JS

CLIENT SIDE TECHNOLOGIES

Introduction to JavaScript

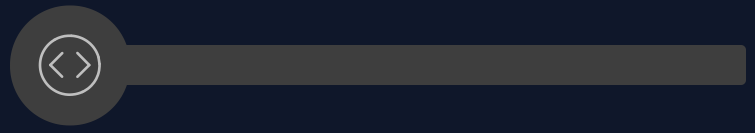
From browser scripting to interactive web experiences

Created by Ahmed Samir

JAVASCRIPT...

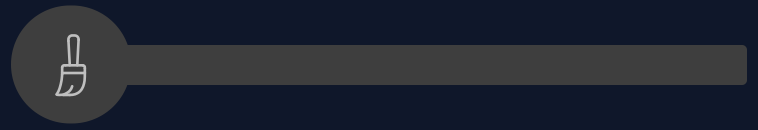
Web Technology Layers

A simple stack showing how structure, styling, and behavior work together



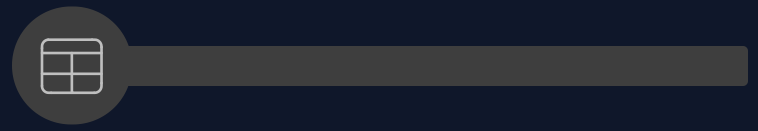
JavaScript

Behavior and interactivity. Handles user actions, dynamic updates, and page logic.



CSS

Styling and presentation. Controls colors, layout, spacing, and visual design.



HTML

Structure and content. Builds the page foundation and organizes information.

JavaScript sits on top because it brings the page to life with interaction and behavior.

JAVASCRIPT...

JavaScript is a web client side scripting language.

Runs in the browser to create interactive, dynamic experiences.

⚡

Web Programming

Server and client work together to deliver interactive web experiences.

Server-Side Programming

- Handles data processing
- Manages databases
- Generates web documents
- Sends HTML to browser

Client-Side Programming

- Runs in the browser
- Handles user interactions
- Validates input
- Creates dynamic effects

🔁 Server builds the content, and the browser uses JavaScript to make it interactive.

Web Programming (Cont.)

How web documents are created and delivered

Servers create web documents

Generate HTML content for requests

Documents are specified in HTML

HTML defines the structure and content

Documents are sent to the browser

The server delivers the document to the client

Browser interprets and displays them

The browser renders the web page for the user

🔁 Server builds the content, and the browser uses JavaScript to make it interactive.

ARCHITECTURE

Web Programming (Cont.)

Your Website Architecture

Client Browser

Renders the website and runs the user interface

Web Server

Handles requests and delivers web assets

HTML, CSS, and JS

Files served to the browser for structure, style, and behavior

JavaScript in Browser

Client-side application logic updates the page dynamically

HTTP requests and responses connect the browser and server, while JavaScript runs in the browser to update the page.

INTRODUCTION TO JAVASCRIPT

HTML

The Foundation of Web Pages

Markup Language

HTML is the markup language for WWW documents

Structure

Provides structure and semantic meaning

Styling

Works with CSS for styling

Interactivity

Works with JavaScript for interactivity

JavaScript

JS

- Simple and flexible
- Lightweight
- Interpreted language
- Runs in the browser
- Enables interactivity
- Dynamic content updates

Client Side Scripting Languages

Modern comparison of the main options

JavaScript (1.x)

Primary language for the web

VBScript

Legacy Microsoft scripting

JScript

Microsoft's JavaScript variant

Other alternatives

TypeScript, Action Script, and more

Markup vs. Scripting vs. Programming Languages

Markup Languages

- HTML
- XML
- Static structure

Scripting Languages

- JavaScript
- VBScript
- Dynamic behavior

Programming Languages

- Java
- C++
- Complex applications

JavaScript vs. Java

JavaScript

- Lightweight
- Interpreted
- Runs in browser
- Dynamic typing
- Prototype-based
- Easy to learn

Java

- Compiled
- Runs on JVM
- Static typing
- Class-based
- More complex
- Enterprise applications

JavaScript History

1995

Created by Brendan Eich at Netscape

1996

Standardization began

1997

ECMAScript standard released

2000s

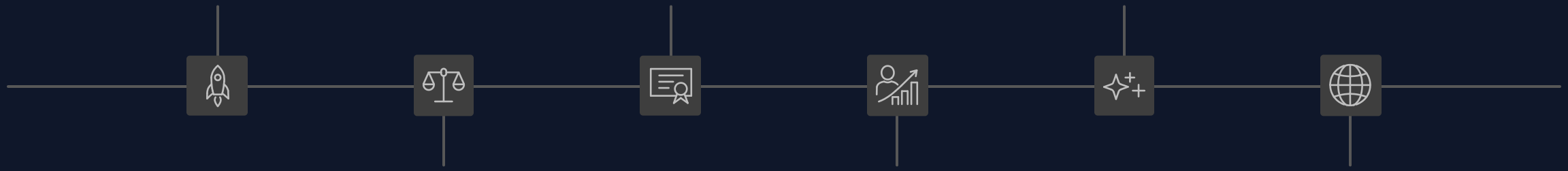
Rapid evolution

2015

ES6 major update

Present

Modern JavaScript



The language evolved from a browser scripting tool into a modern, versatile platform.

JavaScript History (Cont.)

ECMAScript Standards

ES1 (1997)

Original standard

ES3 (1999)

Major improvements

ES5 (2009)

Modern features

ES6 / ES2015

Major overhaul

ES2016+

Annual updates

ECMAScript evolved from the original JavaScript standard into a continuously updated modern language specification.

SERVER-SIDE JAVASCRIPT...

Using NodeJS (a server-side JavaScript environment), we can create server-side, standalone tools and applications of all kinds.

Server-side applications

Command-line tools

Real-time applications

Full-stack JavaScript development

JavaScript Characteristics

- Case sensitive
- Weakly typed
- Interpreted
- Object-oriented
- Functional programming support
- Dynamic
- Flexible syntax

What JavaScript Can Do?

- Give the user more control over the browser
- Validate form data
- Create dynamic HTML content
- Detect browser type and version
- Create cookies
- Perform calculations
- Detect user actions (clicks, typing, etc.)
- Create animations and effects

What JavaScript Can't Do?

Directly access files on the user's system

Access files on other servers

Modify files on the server

Directly access the operating system

Run on the server (client-side only)

Access hardware directly

Bypass security restrictions

JavaScript Strength & Weakness

- Easy to learn
- Lightweight
- Fast execution
- Flexible
- Wide browser support
- Rich ecosystem

- Limited file access
- Security restrictions
- Browser compatibility issues
- No multithreading
- Limited performance for heavy tasks
- Debugging can be difficult

How to Embed JavaScript Code?

Inline <script> Tags

- Write JavaScript anywhere in the HTML
- Code runs when the page loads
- `<script>alert('Hello');</script>`

Event Handler Attributes

- JavaScript code in HTML attributes
- Runs when an event occurs
- `onclick="doSomething()"`

External File (src attribute)

- JavaScript in separate .js file
- Reusable across multiple pages
- Better for organization

2. EVENT HANDLER ATTRIBUTES

How to Embed JavaScript Code? (Cont.)

Event Handler Attributes

JavaScript in HTML Attributes

JavaScript code can be written directly in event handler attributes on an HTML element.

Runs on Events

It executes when the specified event occurs, such as a click or page load.

Simple Example

`onclick="doSomething()"`

Common Events

`onclick`, `onload`, `onmouseover`, `onchange`

Inline Event Handlers

They provide a direct response to user actions in the page.

Direct Interaction

Best for quick actions that respond immediately to what the user does.

3. EXTERNAL FILE (SRC ATTRIBUTE)

How to Embed JavaScript Code? (Cont.)

External File (src Attribute)

JavaScript in separate .js file

Keep your JavaScript in an external file instead of writing it directly in the HTML page.

Link with the src attribute

Use `<script src="file.js"></script>` to connect the HTML document to the script file.

Reusable across multiple pages

The same file can be used on more than one page, reducing repetition.

Better organization and maintenance

Separating code makes projects easier to manage as they grow.

Cleaner HTML code

Your HTML stays simpler and easier to read without embedded JavaScript blocks.

Easier to debug and update

Fixes and changes can be made in one place without editing every page.

Best practice for larger projects

External scripts are the preferred approach for scalable web development.

Outputting Text with JavaScript

Common ways to display or set text in the browser and developer tools.

document.write()

Writes text directly to the document.

Use case: quick page output or simple demos.

innerHTML

Modifies the content inside an element.

Use case: updating page sections dynamically.

console.log()

Outputs messages to the browser console.

Use case: debugging and inspecting values.

alert()

Displays a pop-up dialog box with text.

Use case: simple notifications and prompts.

textContent

Sets plain text content inside an element.

Use case: safe text updates without HTML.

Use **innerHTML** for formatted content, **textContent** for plain text, and **console.log()** for debugging.

Variables

Naming Rules and Basics

First character

Must be a letter, underscore (_), or dollar sign ($).

Example: `name`, `_count`, `$price`

Following characters

Can be letters, digits, underscores, or dollar signs.

Example: `user1`, `total_cost`, `value$`

Case-sensitive

Variable names are case-sensitive.

Example: `firstName` is different from `firstname`

Reserved keywords

Cannot use reserved keywords as variable names.

Example: avoid names like `let` or `class`

Meaningful names

Choose clear names that describe the value.

Example: `totalPrice` instead of `x`

camelCase

Start with a lowercase letter and capitalize each new word.

Example: `firstName`, `lastName`, `masterCard`

Use **camelCase** for readable variable names, and keep names short but descriptive.

Variables (cont.)

Declaration and Initialization

Declaration

Use the keyword **var**, **let**, or **const**.

Example: `var name;`

Initialization

Assign a value to the variable.

Example: `name = "John";`

Declaration + Initialization

Declare and assign in one statement.

Example: `var name = "John";`

Key rules

- Variables are case-sensitive.
- Variables can be declared multiple times.
- Variables can be reassigned, except **const**.

GLOBAL VS. LOCAL

Variables – Life Time & Scope

Global vs. Local

Global Variables

- Declared outside functions
- Accessible everywhere
- Lifetime: entire script execution

Example: `var globalVar = 10;`

Local Variables

- Declared inside functions
- Only accessible within the function
- Lifetime: only while function runs

Example: `function test() { var localVar = 5; }`

1 Scope boundary

Global variables live outside the function block and can be used throughout the script.

2 Inside function

Local variables stay within the function and disappear when execution ends.

Variables – Life Time & Scope (Cont.)

Understanding Scope in Functions

Local Variables in Functions

- Variables declared inside a function
- Only accessible within that function
- Destroyed when function ends

Example:

```
function test() {
  var x = 5;
}
```

Function Parameters

- Parameters are local variables
- Accessible throughout the function
- Receive values from function calls

Example:

```
function add(a, b) {
  return a + b;
}
```

Nested Functions

- Inner functions can access outer variables
- Outer functions cannot access inner variables
- Creates closures

Example:

```
function outer() {
  var x = 1;
  function inner() {
    return x;
  }
}
```

Best Practices

- Keep variables in smallest scope possible
- Avoid global variables
- Use const by default, let when needed

Hoisting

Variable and Function Declaration Behavior

Variables

- Hoisting moves all declarations to the top of their scope
- Initialization stays in place
- `var` declarations are hoisted
- `let` and `const` have temporal dead zone

Functions

- Function declarations are fully hoisted
- Can be called before declaration
- Function expressions are not hoisted

Before hoisting

```
console.log(x);
var x = 5;
```

After hoisting

```
var x;
console.log(x);
x = 5;
```

Function example

```
sayHi();
function sayHi() {
  console.log("Hi");
}
```

Best practice: declare variables at the start of every scope.

INTRODUCTION TO JAVASCRIPT

Strict Mode

Enforce Stricter Parsing and Error Handling

Declaration

- Add `"use strict";` at the start of a script or function
- Must be the first statement
- Example: `"use strict";`

Benefits

- Prevents undeclared variables
- Makes `eval()` safer
- Disables `with` statement
- Improves performance
- Prepares for future JavaScript versions

Scope

- Global strict mode: affects entire script
- Function strict mode: affects only that function
- Can be mixed in the same file

NEW ES6 FEATURE

Variables – Block Scope with let

ES6 Feature for Better Scoping

Before ES6

- JavaScript had no block scope
- Only function scope existed
- Variables leaked out of blocks
- if/for blocks didn't create scope

With let (ES6)

- Block scope: variables limited to { } block
- Prevents variable hoisting issues
- Safer than var
- Example: `let x = 5;` inside block

Comparison

- `var`: function scope
- `let`: block scope
- `const`: block scope + immutable

Code Example

```
function letTest() {
  let x = 1;
  if (true) {
    let x = 2;   // different variable
    console.log(x); // 2
  }
  console.log(x); // 1
}
```

Use `let` when you need variables to stay inside a block.

Key Idea

Block scope helps avoid unexpected variable reuse and makes code easier to reason about.

Variables – Block Scope with let (Cont.)

Practical Examples of Block Scope

for Loop Block Scope

- Each iteration gets its own variable
- `for (let x = 0; x < 10; x++)` creates fresh x each time
- Prevents variable pollution

if Block Scope

- Variables declared in if block stay in that block
- `if (condition) { let x = 5; }` - x only exists here
- Prevents accidental reuse

Comparison: var vs let

- `var`: function scope, hoisted
- `let`: block scope, temporal dead zone
- `let` is safer and more predictable

Practical Benefits

- Prevents variable shadowing issues
- Makes code easier to reason about
- Reduces bugs from variable reuse
- Cleaner, more maintainable code

Code Example

```
for (let x = 0; x < 3; x++) {
  console.log(x);
}
// 0, 1, 2

if (true) {
  let x = 5;
  console.log(x);
}
// x is not available here
```

Use `let` when a variable should stay inside a block.

Why It Matters

Block scope helps prevent accidental reuse, keeps values isolated, and makes JavaScript code easier to maintain.

INTRODUCTION TO JAVASCRIPT

Variables – Constants

Immutable Values with const

JS

Declaration

- Declared with const keyword
- Must be initialized at declaration
- Example: `const PI = 3.14159;`

Immutability

- Cannot be reassigned after initialization
- Cannot be redeclared
- Prevents accidental changes
- Throws error if you try to reassign

Block Scope

- const has block scope (like let)
- Limited to `{ }` block
- Safer than var

Best Practices

- Use const by default
- Use let when you need to reassign
- Avoid var in modern JavaScript

Code Examples

```
const PI = 3.14159;
PI = 3.14; // SyntaxError

const MAX_ITEMS = 10;
MAX_ITEMS++; // SyntaxError

const settings = { theme: "dark" };
```

JavaScript Guidelines

Best Practices for Writing Clean Code

JS

Case Sensitivity

- JavaScript is case sensitive
- myVar is different from myvar
- Be consistent with naming

Naming Conventions

- Use camelCase for variables and functions
- Use UPPER_CASE for constants
- Use PascalCase for classes

Code Organization

- Group related code together
- Use meaningful variable names
- Keep functions small and focused

Comments and Documentation

- Write clear comments
- Explain the why, not the what
- Keep comments up to date

Performance

- Avoid global variables
- Use const by default
- Minimize DOM manipulation

SPECIAL CHARACTERS

JavaScript Guidelines (Cont.)

Special Characters and Escape Sequences

JS

Character	Description
\n	Inserts a new line
\t	Inserts a tab space
\r	Inserts a carriage return
\\	Inserts a backslash
\"	Inserts a double quote
\'	Inserts a single quote
\b	Inserts a backspace
\f	Inserts a form feed

Operators

Types of Operators

JS

Unary Operators

Require one operand

- x++
- ++x
- !x

Binary Operators

Require two operands

- x + y
- x - y
- x * y

Ternary Operator

Requires three operands

- condition ? value1 : value2

ARITHMETIC OPERATORS (y = 5)

Operators (Cont.) — Arithmetic

JS

+ Addition x + y = 15	- Subtraction x - y = 5	* Multiplication x * y = 50	/ Division x / y = 2
% Modulus x % y = 0	++ Increment x++	-- Decrement x--	** Exponentiation x ** y

ASSIGNMENT OPERATORS (x = 10, y = 5)

Operators (Cont.) — Assignment

Assigning Values to Variables

JS

Symbol	Description	Example	Result
=	assignment	x = y	x = 5
+=	add and assign	x += y	x = 15
-=	subtract and assign	x -= y	x = 5
*=	multiply and assign	x *= y	x = 50
/=	divide and assign	x /= y	x = 2
%=	modulus and assign	x %= y	x = 0
**=	exponentiation and assign	x **= y	x = 100000

COMPARISON OPERATORS

Operators (Cont.) — Comparison

Comparing Values

Symbol	Description	Example	Notes
==	Equal to	5 == "5"	Compares value only
!=	Not equal to	5 != 3	Compares value only
===	Strict equal to	5 === "5"	Compares value and type
!==	Strict not equal to	5 !== "5"	Compares value and type
>	Greater than	7 > 3	Left value is larger
<	Less than	2 < 8	Left value is smaller
>=	Greater than or equal to	5 >= 5	True if equal or larger
<=	Less than or equal to	4 <= 6	True if equal or smaller

Use == for value comparison, but prefer === when you need to compare both value and type.

Tip: == and === may look similar, but === is stricter because it checks both the value and its data type.

BITWISE OPERATORS

Operators (Cont.) — Bitwise

Operating on Binary Representations

Bitwise operators work directly on binary values and are less commonly used in modern JavaScript.

Symbol	Name	Description	Example
&	AND	Both bits must be 1	5 & 3
|	OR	At least one bit must be 1	5 | 3
^	XOR	Bits must be different	5 ^ 3
~	NOT	Inverts all bits	~5
<<	Left shift	Shifts bits left	5 << 1
>>	Right shift	Shifts bits right	5 >> 1
>>>	Unsigned right shift	Shifts bits right, fills with 0s	5 >>> 1

Operators (Cont.) — Logical

Combining Conditions

Symbol	Name	Description	Example	Truth / Use
&&	**AND**	Both conditions must be true	(x > 5 && y < 10)	True only if **both** operands are true
||	**OR**	At least one condition must be true	(x > 5 || y < 10)	True if **either** operand is true
!	**NOT**	Reverses the condition	!(x > 5)	Flips true to false, and false to true

Practical use: combine conditions to control access, validate input, or make decisions in code.

Operators (Cont.) — String Concatenation

Combining Strings

Using the + Operator

Concatenate strings with **+**

Example: `"Hello" + " " + "World"`

Result: `"Hello World"`

Using += Operator

Add to existing string

Example: `str += " more text"`

Modifies the original variable

Using Template Literals (ES6)

Use backticks and `${}` for variables

Example: `` `Hello ${name}` ``

More readable and flexible

Using concat() Method

String method to join strings

Example: `"Hello".concat(" ", "World")`

Returns new string

Practical use: build messages, format output, and combine user input in code.

Operators (Cont.) — Special Operators

Advanced Operator Techniques

Conditional (Ternary) Operator

condition ? value1 : value2

Shorthand for if/else

Example: `age >= 18 ? "Adult" : "Minor"`

Comma Operator

Evaluates multiple expressions

Returns the last value

Example: `(x = 1, y = 2, x + y)`

typeof Operator

Returns the type of a value

Example: `typeof "hello"` returns `"string"`

Useful for type checking

instanceof Operator

Checks if object is instance of class

Example: `arr instanceof Array`

Returns true or false

in Operator

Checks if property exists in object

Example: `"name" in person`

Returns true or false

Practical use: write cleaner conditions, inspect values, and test object structure in real code.

Operators (Cont.) — Comma Operator

Evaluating Multiple Expressions

What is the Comma Operator?

- Evaluates multiple expressions
- Returns the value of the last expression
- Useful for compact code

Syntax

`expression1, expression2, expression3, ...`

Example: `(x = 1, y = 2, x + y)`

Returns **3** (the result of `x + y`)

Common Uses

- for loop initialization: `for (i = 0, j = 10; i < j; i++, j--)`
- Multiple assignments in one statement
- Compact conditional logic

Important Notes

- All expressions are evaluated
- Only the last value is returned
- Can make code harder to read
- Use sparingly for clarity

Practical example: `var k = 0, i, j = 0;` and `(x = 1, y = 2, x + y)`.

INTRODUCTION TO JAVASCRIPT

Operators (Cont.) — typeof Operator

Checking Data Types

What is typeof?

- A unary operator that returns a string
- Tells you the data type of a value
- Useful for type checking and validation

Return Values

- "string" - for strings
- "number" - for numbers
- "boolean" - for booleans
- "object" - for objects and arrays
- "undefined" - for undefined values
- "function" - for functions
- "symbol" - for symbols (ES6)

</>

Examples

`typeof "hello"` returns `"string"`

`typeof 42` returns `"number"`

`typeof true` returns `"boolean"`

`typeof {}` returns `"object"`

`typeof undefined` returns `"undefined"`

`typeof function() {}` returns `"function"`

Use Cases

- Validate user input
- Check function parameters
- Conditional logic based on type

INTRODUCTION TO JAVASCRIPT

Coercion

Automatic Type Conversion

What is Coercion?

- Converting from one data type to another
- Happens automatically in expressions
- JavaScript is weakly typed

Examples

```
"5" + 3 = "53"   // string concatenation
"5" - 3 = 2     // numeric subtraction
true + 1 = 2    // boolean to number
null + 5 = 5    // null to 0
```

Implicit vs Explicit

- **Implicit:** automatic conversion
- **Explicit:** manual conversion with Number(), String(), Boolean()

Best Practices

- Be aware of coercion rules
- Use strict equality (===) to avoid surprises
- Explicitly convert when needed

INTRODUCTION TO JAVASCRIPT

Operators Precedence

Order of Operations in JavaScript

What is Operator Precedence?

- Determines the order in which operators are evaluated
- Higher precedence operators are evaluated first
- Similar to PEMDAS in math

Common Precedence Order (High to Low)

- Parentheses ()
- Exponentiation **
- Multiplication, Division, Modulus *, /, %
- Addition, Subtraction +, -
- Comparison <, >, <=, >=
- Equality ==, !=, ===, !==
- Logical AND &&
- Logical OR ||
- Assignment =, +=, -=, etc.

Examples

```
2 + 3 * 4 = 14   // not 20
(2 + 3) * 4 = 20 // parentheses override
```

Best Practice

- Use parentheses to make intent clear
- Don't rely on precedence rules

Controlling Program Flow

Decision Making and Loops

Conditional Statements

Make decisions based on conditions.

- if ... else
- switch ... case

Loop Statements

Repeat code blocks.

- for
- while
- do ... while

Controlling Statements (Cont.)

Conditional Statements

if ... else

Execute code if condition is true

- Optional else block for false condition
- Example: `if (x > 5) { ... } else { ... }`

else if

Test multiple conditions

- Executed in order
- Example: `if (x > 10) { ... } else if (x > 5) { ... }`

switch ... case

Test one value against multiple cases

- More efficient than multiple if/else
- Example: `switch (day) { case 1: ... break; }`

Ternary Operator

Shorthand for if/else

- `condition ? value1 : value2`
- Example: `age >= 18 ? "Adult" : "Minor"`

2. LOOP STATEMENTS

Controlling Statements (Cont.)

Loop Statements

for loop

Repeat code a specific number of times

- Syntax: `for (init; condition; increment) { ... }`
- Example: `for (let i = 0; i < 10; i++) { ... }`

while loop

Repeat while condition is true

- Syntax: `while (condition) { ... }`
- Example: `while (x < 10) { x++; }`

do ... while loop

Execute at least once, then repeat while condition is true

- Syntax: `do { ... } while (condition);`
- Example: `do { x++; } while (x < 10);`

for ... in loop

Iterate over object properties

- Syntax: `for (key in object) { ... }`

for ... of loop (ES6)

Iterate over array values

- Syntax: `for (value of array) { ... }`

JS

INTRODUCTION TO JAVASCRIPT

Controlling Statements (Cont.) — Breaking Loops

Loop Control Statements

break statement

- Breaks out of the loop immediately
- Stops execution of the loop

```
if (x === 5) {
  break;
}
```

continue statement

- Skips the current iteration
- Continues with the next iteration

```
if (x === 5) {
  continue;
}
```

Use Cases

- break: exit when condition is met
- continue: skip certain iterations
- Useful for filtering and early exit

Examples

- for loop with break
- while loop with continue
- Nested loops with labeled break

Dialogue Boxes — Alert

Simple User Notifications

Purpose

Display a message to the user

Syntax

`alert("message");`

User interaction

User must click OK to dismiss

Use case

Simple notifications and warnings

Example code

```
<script>
  alert("Click OK to continue.");
</script>
```

Alert box

Alert

Click OK to continue.

Dialogue Boxes (Cont.) — Prompt

Getting User Input

Purpose

- Get input from the user
- Display a text input field
- User can type and submit

Syntax

`prompt("message", "default value");`

Returns the user's input as a string

Returns `null` if user clicks Cancel

Use Cases

- Ask for user's name
- Get search terms
- Request confirmation with text input

Example

```
var name = prompt("What is your name?", "John");
if (name !== null) {
  ...
}
```

Dialogue Boxes (Cont.) — Confirm

Getting Yes/No Confirmation

Purpose

- Ask user for confirmation
- Display OK and Cancel buttons
- Returns boolean value

Syntax

`confirm("message");`

Returns `true` if user clicks OK

Returns `false` if user clicks Cancel

Use Cases

- Confirm before deleting
- Ask for permission
- Verify important actions

Example

```
if (confirm("Are you sure?")) { ... }
```

Useful for destructive operations

JavaScript Built-in Functions

Common Functions You Can Use

String Functions

- charAt() — get a character at a position
- substring() — extract part of a string
- indexOf() — find the first match
- toUpperCase() — convert to uppercase
- toLowerCase() — convert to lowercase

Number Functions

- parseInt() — convert string to integer
- parseFloat() — convert string to decimal
- isNaN() — check for invalid numbers
- Math.round() — round to nearest integer
- Math.floor() — round down

Array Functions

- push() — add to the end
- pop() — remove from the end
- shift() — remove from the start
- unshift() — add to the start
- slice() — copy part of an array
- splice() — add or remove items

Object Functions

- Object.keys() — return property names
- Object.values() — return property values
- Object.entries() — return key-value pairs

JavaScript Built-in Functions (Cont.)

More Essential Functions

Date and Time Functions

- Date() — create date objects
- getTime() — get milliseconds since epoch
- getFullYear() — get year
- getMonth() — get month (0-11)

Math Functions

- Math.abs() — absolute value
- Math.sqrt() — square root
- Math.pow() — exponentiation
- Math.min() / Math.max() — find min/max
- Math.random() — random number 0-1

String Methods

- split() — split string into array
- replace() — replace text
- trim() — remove whitespace
- includes() — check if contains

INTRODUCTION TO JAVASCRIPT

JavaScript Built-in Functions (Cont.)

Advanced Built-in Functions

Array Methods

- map() — transform each element
- filter() — keep elements that match a condition
- reduce() — combine elements into a single value
- find() — find the first matching element
- forEach() — execute a function for each element

JSON Functions

- JSON.stringify() — convert an object to a JSON string
- JSON.parse() — convert a JSON string to an object

Global Functions

- parseInt() — convert a string to an integer
- parseFloat() — convert a string to a decimal
- isNaN() — check if a value is not a number
- encodeURI() — encode a URL
- decodeURI() — decode a URL

JavaScript User-Defined Functions

Creating Custom Functions

Function declaration syntax

```
function dosomething(x) {
  statements
}
```

Parameters are placed inside the parentheses.

Function call syntax

```
dosomething("hello");
```

Arguments are passed when the function is called.

Return values

```
function sayHi() {
  statements
  return "hi";
}
```

Functions can return a value for reuse later.

Benefits of using functions

- Reuse code instead of repeating it
- Make programs easier to read
- Organize logic into smaller parts
- Return results from calculations

Key concepts: A function can take parameters, receive arguments, perform statements, and optionally return a value.

JavaScript User-Defined Functions (Cont.)

Advanced Function Concepts

Function Hoisting

- Functions can be called before declaration
- Function declarations are hoisted
- Function expressions are not hoisted

```
sayHello();

function sayHello() {
  console.log("Hello!");
}
```

Scope and Closures

- Functions create their own scope
- Inner functions can access outer variables
- Closures remember their scope

```
function outer() {
  let message = "Hi";
  return function inner() {
    console.log(message);
  };
}
```

Recursion

- Functions can call themselves
- Must have a base case to stop
- Useful for tree/graph traversal

```
function factorial(n) {
  if (n === 0) return 1;
  return n * factorial(n - 1);
}
```

Arrow Functions (ES6)

- Shorter syntax: const add = (a, b) => a + b;
- Lexical this binding
- Great for callbacks

```
const add = (a, b) => a + b;
const nums = [1, 2, 3].map(n
=> n * 2);
```

JS

TYPES OF ERRORS

JavaScript Debugging Errors

Types of Errors

Syntax Errors

Mistakes in code structure

- Caught by the parser
- Example: missing semicolon
- Example: wrong brackets

Runtime Errors

Occur during execution

- Code is syntactically correct but fails
- Example: undefined variable
- Example: type mismatch

Logic Errors

Code runs but produces wrong results

- Hardest to find
- Example: wrong calculation
- Example: incorrect condition

JS

INTRODUCTION TO JAVASCRIPT

JavaScript Debugging

Tools and Techniques

Browser Developer Tools

Modern browsers have built-in JavaScript console

- Access via F12 or right-click > Inspect
- View errors and warnings
- Execute code in console

Console Methods

Quick ways to inspect and trace your code

- `console.log()` - log messages
- `console.error()` - log errors
- `console.warn()` - log warnings
- `console.table()` - display data as table

Debugging Techniques

Use the debugger to pause and inspect execution

- Set breakpoints
- Step through code
- Watch variables
- Inspect the DOM

Debugging Errors on FF (Cont.)

Using Browser Developer Tools

Browser Developer Tools

Modern browsers have built-in developer tools

- Firefox: Press F12 or right-click > Inspect
- Chrome: Press F12 or right-click > Inspect
- Safari: Enable Developer Menu in preferences

Console Tab

View JavaScript errors and warnings

- Execute JavaScript code directly
- Use `console.log()` for debugging
- See network requests and responses

Debugger Tab

Pause code and inspect execution step by step

- Set breakpoints in code
- Step through code line by line
- Watch variables and their values
- Inspect the call stack

Elements/Inspector Tab

Edit structure and styles in real time

- View and edit HTML structure
- Inspect CSS styles
- Modify DOM in real-time
- Test layout changes

JS

console Object

Browser Debugging Interface

console.log()

Output general messages

Most commonly used

`console.log("Hello");`

console.error()

Output error messages

Displayed in red

`console.error("Error!");`

console.warn()

Output warning messages

Displayed in yellow

`console.warn("Warning!");`

console.table()

Display data as a table

Great for arrays and objects

`console.table(data);`

console.time()

Measure execution time

Use to benchmark code

`console.time("label");`

JS

Self Study

Continue Your Learning Journey

Build confidence by exploring the topics below and deepening your JavaScript skills one step at a time.

ES.next updates

What's new in later ECMAScript standard versions (ES.next)?

Advanced concepts

Closures, prototypes, and async/await

DOM and events

Build interactive experiences with the page

APIs and libraries

React, Vue, Angular, and modern integrations

Node.js backend

Extend JavaScript beyond the browser

Testing and debugging

Best practices for reliable, maintainable code

Performance

Optimize for speed, responsiveness, and scale

Security

Protect apps with strong JavaScript safety habits

Keep going — every topic you master makes you a stronger JavaScript developer.

Thank You!